

파이썬 기초 프로그래밍과 응용

250217 - 1일차

왜 프로그래밍을 배워야하는가?



왜 프로그래밍을 배워야하는가?



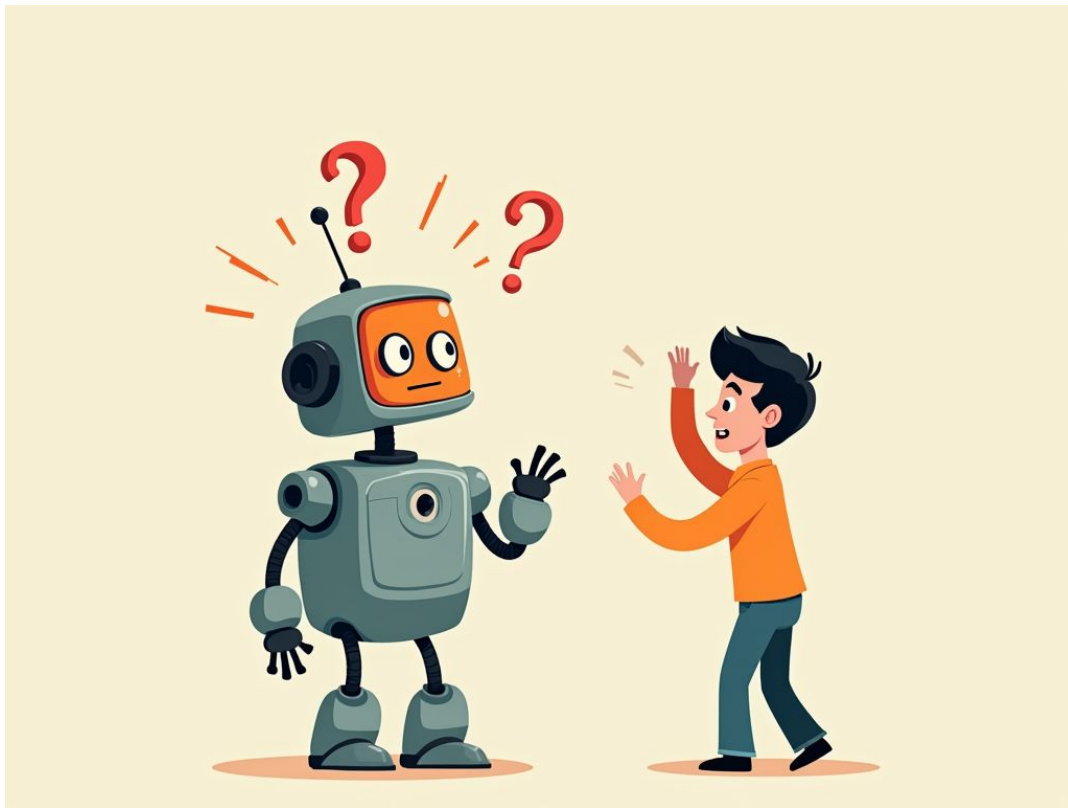
왜 프로그래밍을 배워야하는가?



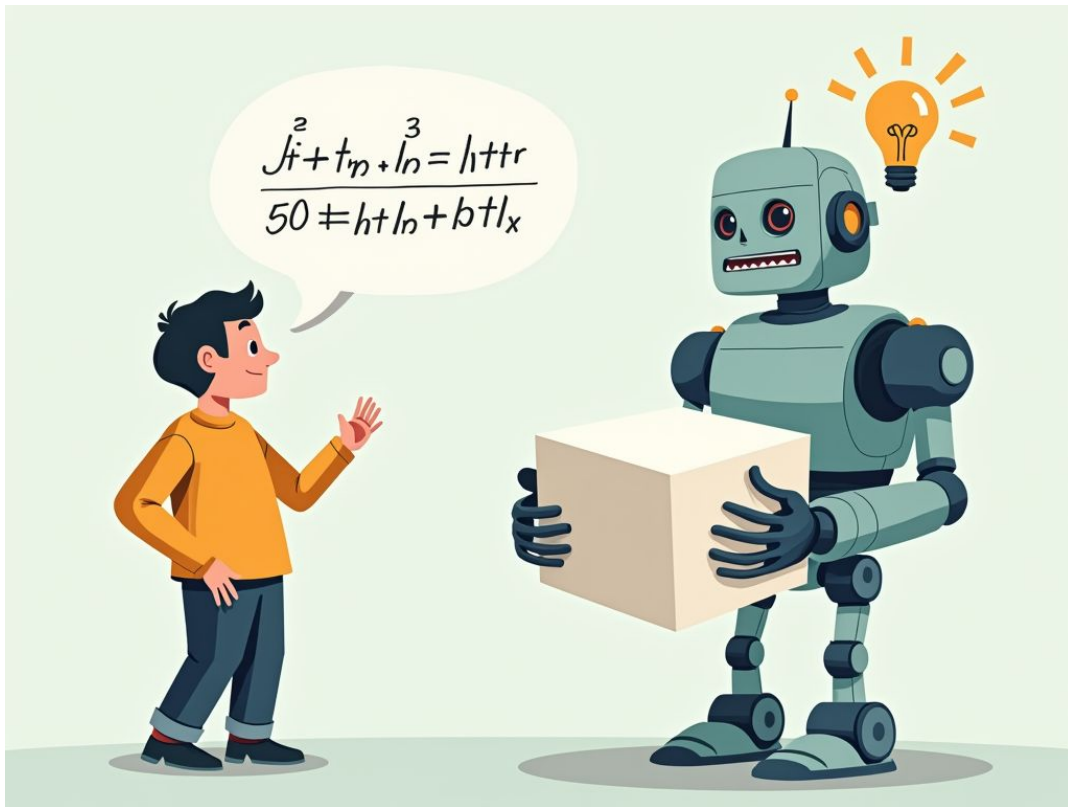
왜 프로그래밍을 배워야하는가?

- 프로그래밍은 데이터를 효과적으로 다루고 분석하기 위한 핵심 기술
- 프로그래밍을 통해
 - 대규모 데이터 처리 가능: 방대한 데이터를 효율적으로 분석하고 의미 있는 패턴을 추출한다.
 - 자동화된 문제 해결: 컴퓨터가 자동으로 문제를 해결하도록 하여 시간과 노력을 절약한다.

왜 프로그래밍을 배워야하는가?



왜 프로그래밍을 배워야하는가?



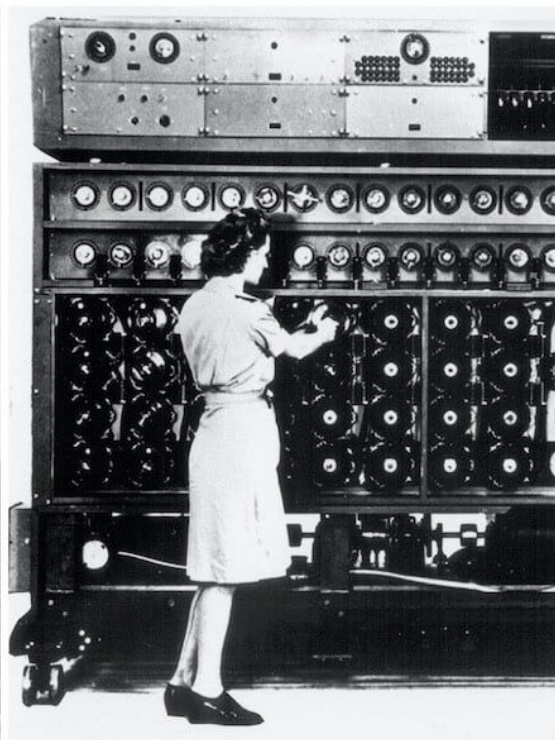
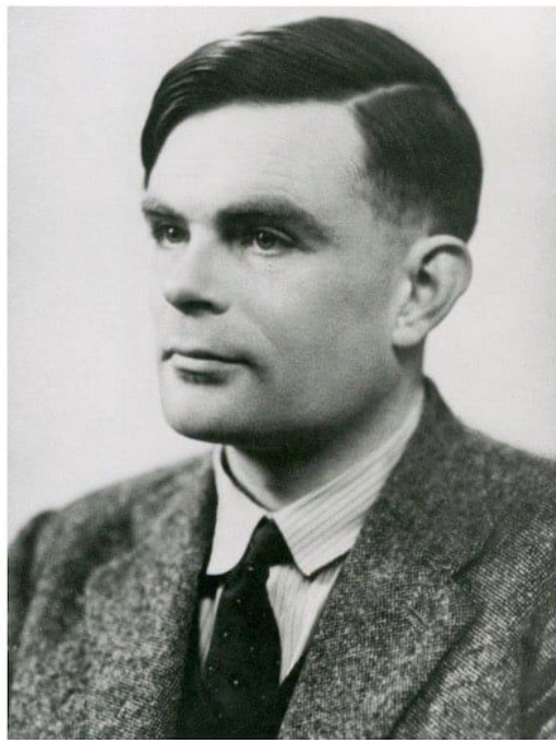
왜 프로그래밍을 배워야하는가?

- 프로그래밍은 인간과 컴퓨터를 이어주는 다리
- 프로그래밍은 컴퓨터와의 소통 언어
 - 컴퓨터에게 정확한 지시를 내리기 위해서는 그들의 언어를 이해가 필요
 - 프로그래밍 언어를 통해 우리는 컴퓨터와 효과적으로 소통 가능

컴퓨터란?



컴퓨터란?

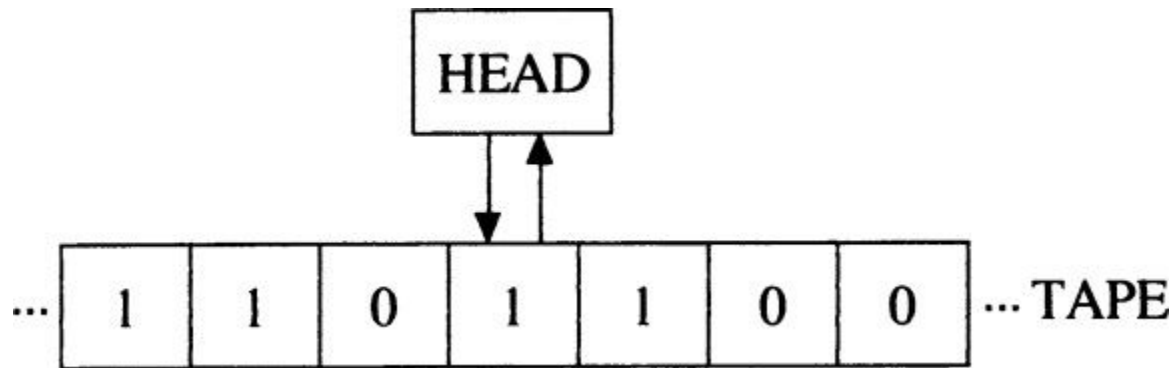


<https://youtu.be/6178TGqHkH0>

컴퓨터란? - vs 인간

- 원래 'computer'는 수학적 계산을 전문적으로 수행하는 직업.
- 에니그마는 1,054,560의 회전자 설정이 존재.
이 설정을 1초에 1개씩 확인한다 해도 293시간이 소요.
- 튜링의 봄베는 17,576 조합을 20분에 처리 가능.
80대 넘는 봄베를 통해 에니그마 암호화를 풀어냄.

컴퓨터란? - 튜링 기계

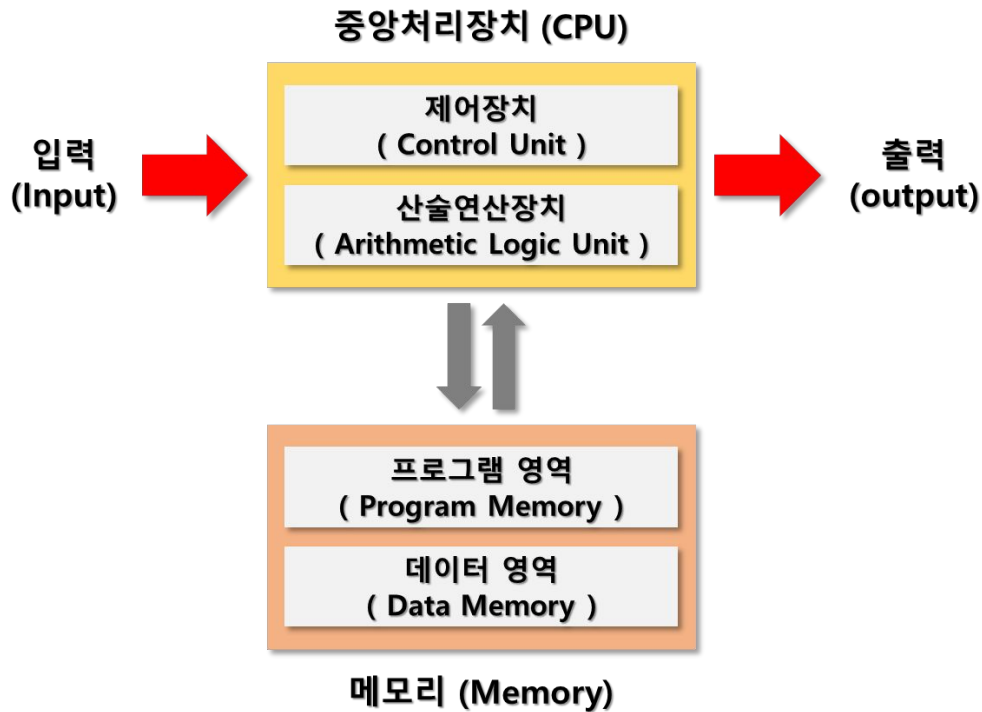
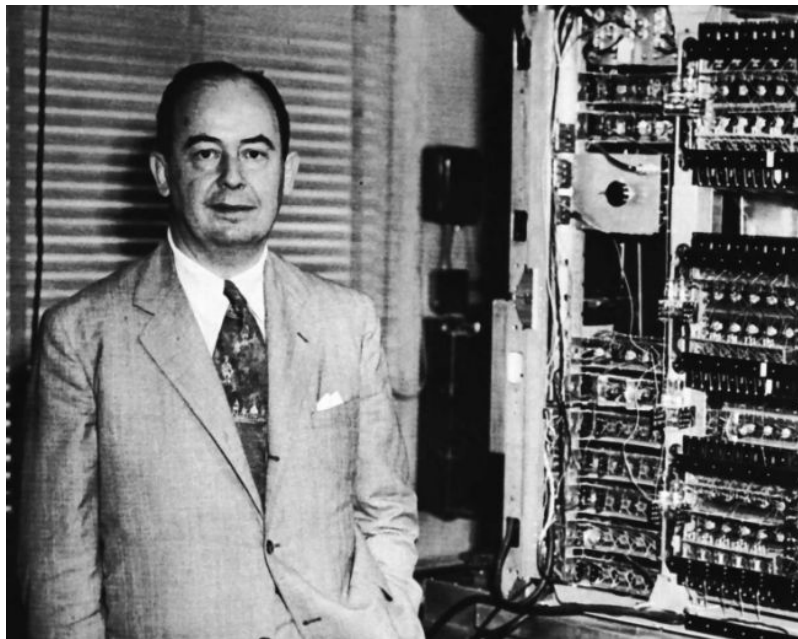


<https://doodles.google/doodle/alan-turings-100th-birthday>

컴퓨터란? - 튜링 기계

- 튜링기계는 무한한 길이의 테이프에 쓰여진 기호들을 일정한 규칙에 따라 읽고 쓸 수 있는 가상의 기계
- 현대의 모든 컴퓨터는 이 튜링기계의 개념을 구현한 물리적 장치

컴퓨터란? - 폰 노이만 컴퓨터 구조



컴퓨터란? - 폰 노이만 컴퓨터 구조

- CPU: 연산장치와 제어장치
- 메모리: 프로그램과 데이터 저장
- 입출력장치: 키보드, 프린터와 같은 데이터 상호작용을 위한 장치
- 시스템 버스: 모든 구성요소 연결

컴퓨터란? - 프로그래밍 언어 계층

 hello.py X

 hello.py > ...

```
1  msg = "Hello World"
2  print(msg)
3  |
```

01001010101

10110010101

10001110100

110010101011

110101010101

컴퓨터란? - 프로그래밍 언어 계층

- 저급 언어 (기계어)
 - 하드웨어 직접 제어
 - CPU가 직접 실행 가능
- 고급 언어 (예: Python)
 - 사람이 이해하기 쉬운 형태
 - 컴파일러/인터프리터로 변환 필요

Python

- 창시자: Guido van Rossum (귀도 반 로섬)
- 영어처럼 읽히는 쉬운 코드



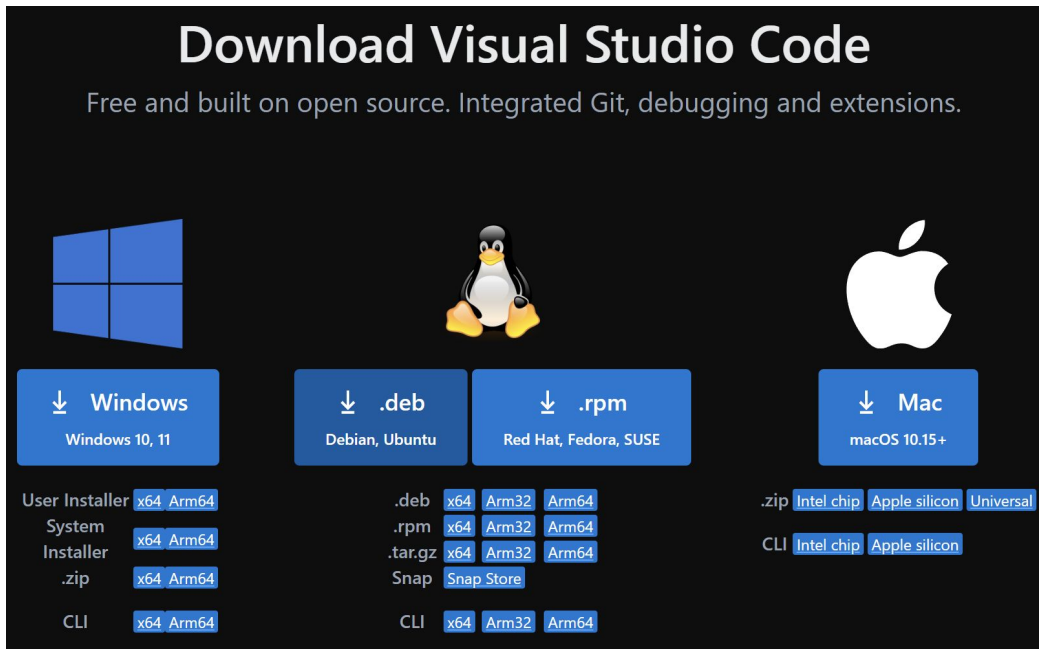
https://en.wikipedia.org/wiki/Guido_van_Rossum

Python 개발 환경 구축하기



<https://apps.microsoft.com/detail/9NRWMJP3717K>

VS Code 설치하기

The image shows the Visual Studio Code download page. At the top, it says "Download Visual Studio Code" in large white letters, followed by the tagline "Free and built on open source. Integrated Git, debugging and extensions." Below this, there are three main sections for different operating systems: Windows, Linux, and Mac. Each section has a large icon (Windows logo, Tux penguin, and Apple logo respectively) and a blue button with a download arrow icon and the OS name. Under the Windows button, there are links for "User Installer", "System Installer", ".zip", and "CLI", each with "x64" and "Arm64" architecture options. Under the Linux section, there are buttons for ".deb" (Debian, Ubuntu) and ".rpm" (Red Hat, Fedora, SUSE). Below these are links for ".deb", ".rpm", ".tar.gz", and "Snap", each with "x64", "Arm32", and "Arm64" architecture options. Under the Mac section, there is a "Mac" button for "macOS 10.15+", and below it are links for ".zip" and "CLI", each with "Intel chip" and "Apple silicon" options. The entire page has a dark background with white and blue text and icons.

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.

Windows
Windows 10, 11

Linux
Debian, Ubuntu (deb)
Red Hat, Fedora, SUSE (rpm)

Mac
macOS 10.15+

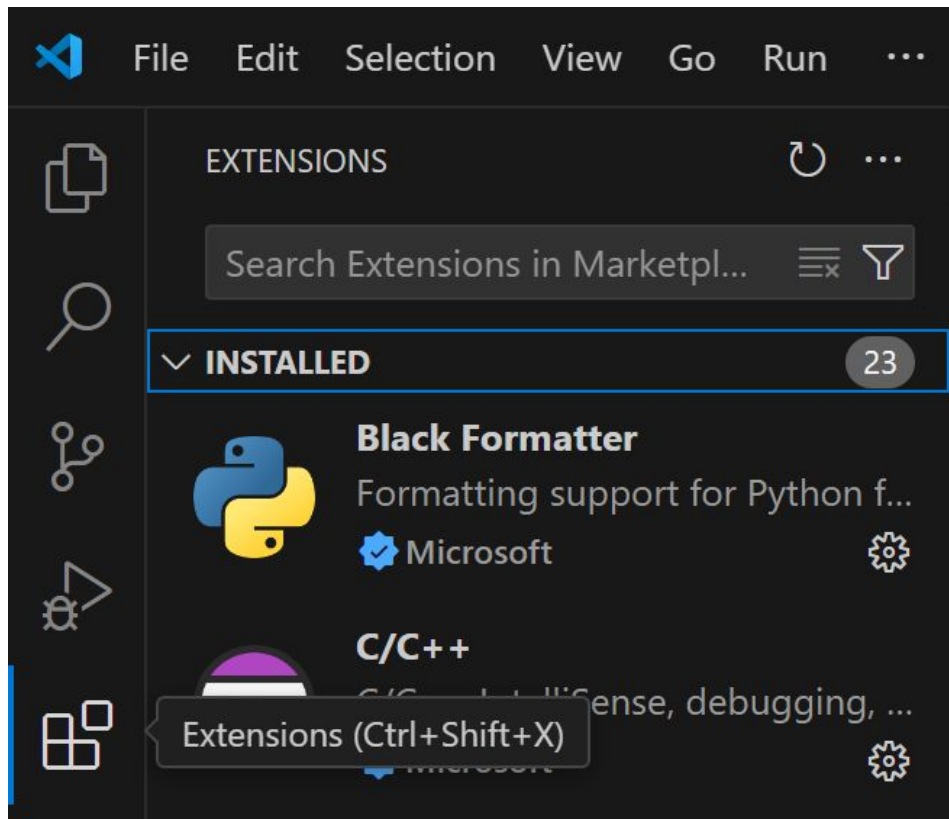
User Installer: x64, Arm64
System Installer: x64, Arm64
.zip: x64, Arm64
CLI: x64, Arm64

.deb: x64, Arm32, Arm64
.rpm: x64, Arm32, Arm64
.tar.gz: x64, Arm32, Arm64
Snap: Snap Store
CLI: x64, Arm32, Arm64

.zip: Intel chip, Apple silicon, Universal
CLI: Intel chip, Apple silicon

<https://code.visualstudio.com/download>

Python 익스텐션 설치하기



Python 익스텐션 설치하기



Python v2024.12.3

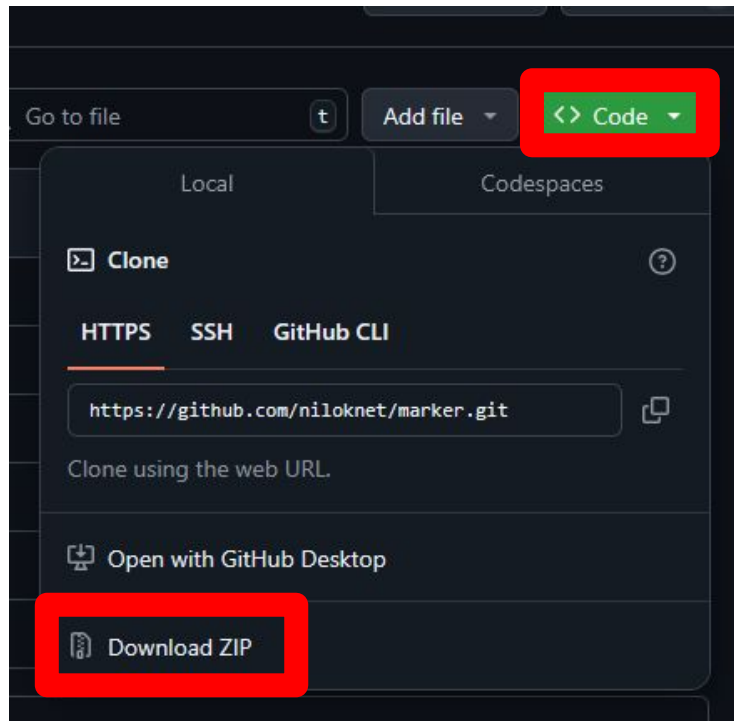
Microsoft  microsoft.com |  133,747,098 | ★★★★★☆ (595)

Python language support with extension access points for IntelliSense (Pylance), Debuggin...

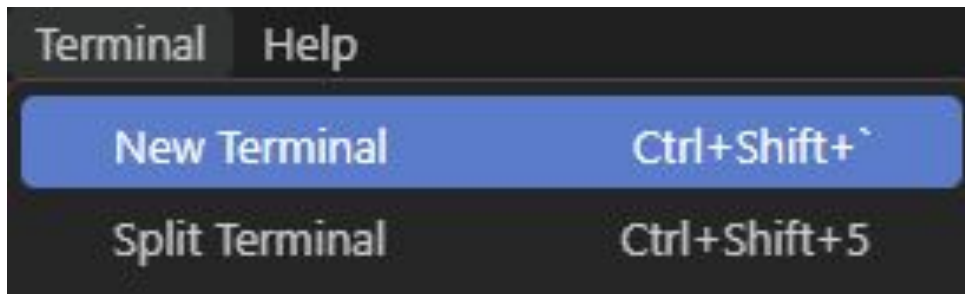
[Disable](#) [Uninstall](#)  [Switch to Pre-Release Version](#) ☒ Auto Update 

OpenAI Gym 환경 확인하기

<https://github.com/niloknet/marker/tree/2025-02>



OpenAI Gym 환경 확인하기



OpenAI Gym 환경 확인하기

```
python --version
```

파이썬 버전 확인

```
python -m venv .venv
```

파이썬 독립된 환경 만들기

```
.\.venv\Scripts\Activate.ps1
```

독립된 환경 활성화

```
pip install -r requirements.txt
```

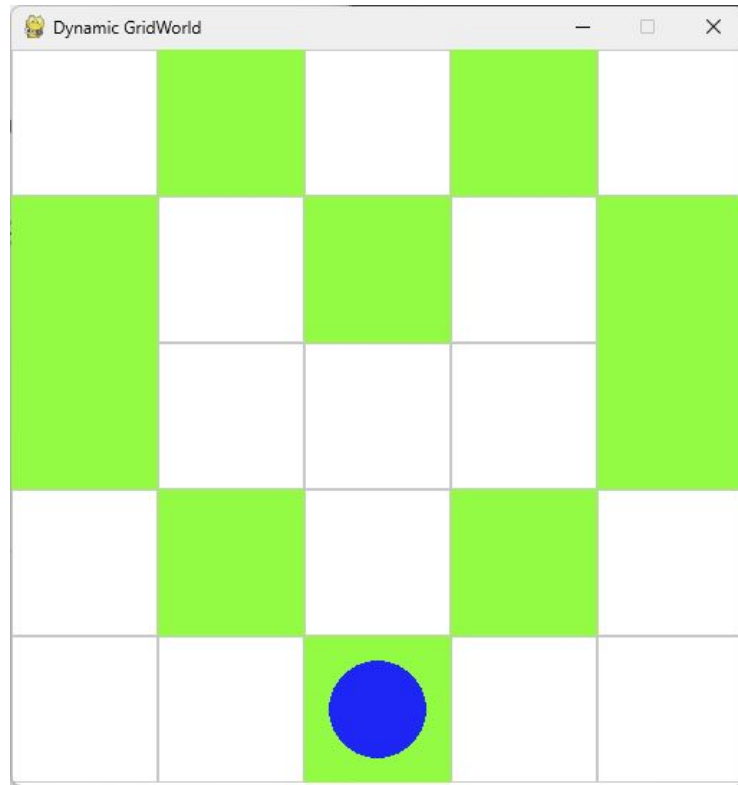
OpenAI Gym 환경 설치

OpenAI Gym 환경 확인하기

- PS D:\marker-main> python --version
Python 3.11.9
- PS D:\marker-main> python -m venv .venv
- PS D:\marker-main> .\.venv\Scripts\Activate.ps1
- (.venv) PS D:\marker-main> pip install -r requirements.txt

OpenAI Gym 환경 확인하기

```
○ (.venv) PS D:\marker-main> python .\0_robot_env.py
```



OpenAI Gym 환경 - 에이전트 이동

`env.move(방향)`

- 방향: 'up', 'down', 'left', 'right' 중 하나
- 지정한 방향으로 에이전트를 이동시킵니다.
- 이동 가능한 경우에만 이동하며, 벽에 의해 막힌 경우 이동하지 않습니다.
- `move_up()`, `move_down()`, `move_left()`, `move_right()` 로도 사용 가능합니다.

OpenAI Gym 환경 - 객체 조작

`env.place()`

- 현재 에이전트 위치에 객체(박스)를 배치합니다.

`env.pick_up()`

- 현재 에이전트 위치에서 객체(박스)를 제거합니다.

OpenAI Gym 환경 - 주변 환경 감지

`env.sense(방향)`

- 방향: 'up', 'down', 'left', 'right' 중 하나
- 반환값: {'wall': True/False, 'box': True/False}
- 지정한 방향에 대해 벽과 박스 여부를 감지합니다.
- 반환값을 해당 방향에 있는 벽과 박스의 존재 여부를 나타냅니다.
- `sense`는 `move`와 비슷하게
`sense_up()`, `sense_down()`, `sense_left()`, `sense_right()`로도 사용할 수 있습니다.

OpenAI Gym 환경 - 주변 환경 감지

`env.sense_box(방향)`

- 방향: 'up', 'down', 'left', 'right' 중 하나
- 반환값: True or False
- 지정한 방향에 대해 박스 존재 여부를 감지합니다.
- 반환값을 해당 방향에 있는 박스의 존재 여부를 나타냅니다.
-

`env.sense_wall(방향)`

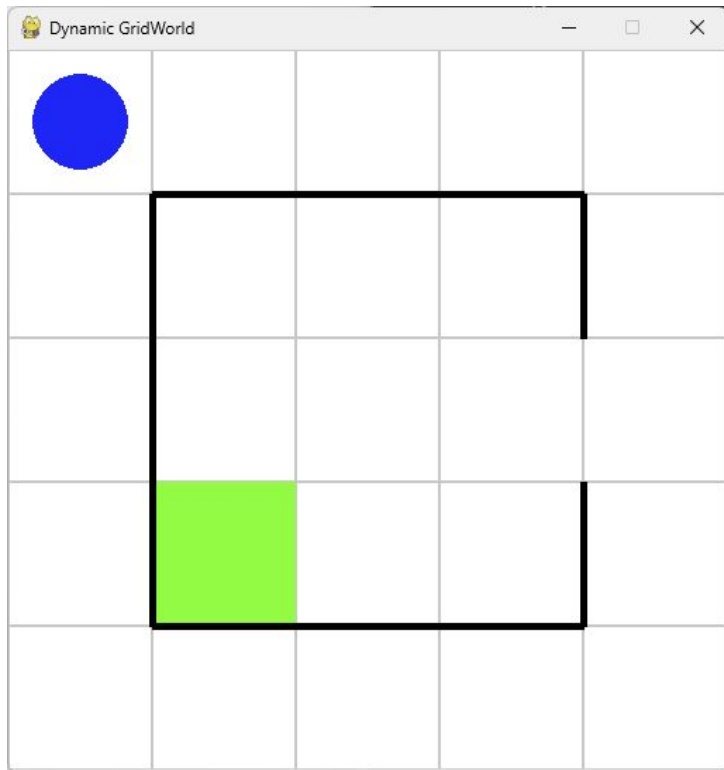
- 방향: 'up', 'down', 'left', 'right' 중 하나
- 반환값: True or False
- 지정한 방향에 대해 벽 존재 여부를 감지합니다.
- 반환값을 해당 방향에 있는 벽의 존재 여부를 나타냅니다.

OpenAI Gym 환경 - 주변 환경 감지

`env.sense_all()`

- 반환값: { 'up': {'wall': True/False, 'box': True/False},
 'down': {'wall': True/False, 'box': True/False},
 'left': {'wall': True/False, 'box': True/False},
 'right': {'wall': True/False, 'box': True/False}}
- 모든 방향의 박스와 벽 정보를 반환합니다.
- `sense_all_box()`, `sense_all_wall()`은 각 방향의 박스와 벽 정보만을 반환합니다.
- 종류를 지정한 전 방향 감지 함수의 반환값은
{ 'up': True/False, 'down': True/False, 'left': True/False, 'right': True/False }
형태입니다.

OpenAI Gym 환경 익숙해지기



변수

```
1  a = 1  
2
```



변수

```
1  alpha = 17
2  bravo = 3.14
3  charlie = 'python'
4  delta = True
5  echo = None
```

많이 사용하는 변수의 종류

- int: 정수(Integer), -1, 0, 1, ...
- float: 실수, 0.3
- str: 문자열, 'Hello'
- bool: 논리 값 (True 또는 False)
- NoneType: None

변수형 - 정수

- 열 손가락으로 셀 수 있는 최대의 수는?
- 8개의 손가락은?
- 16개의 손가락은?
- 64개의 손가락은?

변수형 - 실수

Floating Point Numbers (Single precision) - by IEEE-754 standard



$$\text{number} = (-1)^{\text{sign}} * \text{mantissa} * 2^{\text{exponent}}$$

변수형 - 실수

<https://0.30000000000000004.com>

Python 3

```
print(.1 + .2)
```

AND

```
.1 + .2
```

AND

```
float(decimal.Decimal('.1') + decimal.Decimal('.2'))
```

AND

```
float(fractions.Fraction('0.1') + fractions.Fraction('0.2'))
```

```
0.30000000000000004
```

AND

```
0.30000000000000004
```

AND

```
0.3
```

AND

```
0.3
```

Python (both 2 and 3) supports decimal arithmetic with the [decimal](#) module, and true rational numbers with the [fractions](#) module.

변수형 - 문자열

Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex				
0	00	NUL	16	10	DLE	32	20		48	30	0	64	40	@	80	50	P	96	60	`	112	70	p
1	01	SOH	17	11	DC1	33	21	!	49	31	1	65	41	A	81	51	Q	97	61	a	113	71	q
2	02	STX	18	12	DC2	34	22	"	50	32	2	66	42	B	82	52	R	98	62	b	114	72	r
3	03	ETX	19	13	DC3	35	23	#	51	33	3	67	43	C	83	53	S	99	63	c	115	73	s
4	04	EOT	20	14	DC4	36	24	\$	52	34	4	68	44	D	84	54	T	100	64	d	116	74	t
5	05	ENQ	21	15	NAK	37	25	%	53	35	5	69	45	E	85	55	U	101	65	e	117	75	u
6	06	ACK	22	16	SYN	38	26	&	54	36	6	70	46	F	86	56	V	102	66	f	118	76	v
7	07	BEL	23	17	ETB	39	27	'	55	37	7	71	47	G	87	57	W	103	67	g	119	77	w
8	08	BS	24	18	CAN	40	28	(	56	38	8	72	48	H	88	58	X	104	68	h	120	78	x
9	09	HT	25	19	EM	41	29	)	57	39	9	73	49	I	89	59	Y	105	69	i	121	79	y
10	0A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	J	90	5A	Z	106	6A	j	122	7A	z
11	0B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	K	91	5B	[	107	6B	k	123	7B	{
12	0C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	L	92	5C	\	108	6C	l	124	7C	
13	0D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	M	93	5D	]	109	6D	m	125	7D	}
14	0E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	^	110	6E	n	126	7E	~
15	0F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	_	111	6F	o	127	7F	DEL

변수형 - 논리값 (Boolean)

Boolean Operators

AND

A	B	A AND B
True	True	True
True	False	False
False	True	False
False	False	False

OR

A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

NOT

A	NOT A
True	False
False	True

```

6  # 변수에 현재 위치 저장
7  saved_pos_x = env.agent_pos[0]
8  saved_pos_y = env.agent_pos[1]
9
10 print(f"저장된 x 좌표: {saved_pos_x}, 저장된 y 좌표: {saved_pos_y}")
11 print(f"저장된 좌표: ({saved_pos_x}, {saved_pos_y})")
12
13 # 아래로 이동
14 # env.move_down()
15
16 print(f"저장된 좌표: ({saved_pos_x}, {saved_pos_y})")
17 print(f"현재 좌표: ({env.agent_pos[0]}, {env.agent_pos[1]})")
18
19 # 현재 위치와 이전 위치 비교하여 차이를 변수에 저장
20 difference_x = env.agent_pos[0] - saved_pos_x
21 difference_y = env.agent_pos[1] - saved_pos_y
22
23 print(f"지금과 이전의 좌표 차이 ({difference_x}, {difference_y})")

```

```

(.venv) PS D:\marker-main> python .\1_variables.py
저장된 x 좌표: 0, 저장된 y 좌표: 0
저장된 좌표: (0, 0)
저장된 좌표: (0, 0)
현재 좌표: (1, 0)
지금과 이전의 좌표 차이 (1, 0)
Press 'ESC' or 'ENTER' to close the window to exit.

```

조건문 - 비교문

- `==` : 두 값이 같음
- `!=` : 두 값이 다름
- `>` : 왼쪽 값이 오른쪽 값보다 큼
- `<` : 왼쪽 값이 오른쪽 값보다 작음
- `>=` : 왼쪽 값이 오른쪽 값보다 크거나 같음
- `<=` : 왼쪽 값이 오른쪽 값보다 작거나 같음

조건문 - 비교문

```
1  x = 5
2  y = 3
3
4  if x > y:
5      print("x가 y보다 큼니다.")
```

조건문 - 논리연산

- **and** : 두 값이 모두 참일 때 참
- **or** : 두 값 중 하나라도 참일 때 참
- **not** : 값이 참이면 거짓, 거짓이면 참

Boolean Operators

AND		
A	B	A AND B
True	True	True
True	False	False
False	True	False
False	False	False

OR		
A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

NOT	
A	NOT A
True	False
False	True

조건문 - 논리연산

```
1  x = True
2  y = False
3
4  if x and y:
5      |    print("x와 y가 모두 참입니다.")
6
7  if x or y:
8      |    print("x와 y 중 하나가 참입니다.")
9
10 if x and not y:
11     |    print("x는 참이고 y는 참이 아닙니다.")
```

조건문 - else, elif

```
1  x = 5
2
3  if x > 10:
4      print("x가 10보다 큼니다.")
5  else:
6      print("x가 10보다 작거나 같습니다.")
7
8  if x > 10:
9      print("x가 10보다 큼니다.")
10 elif x > 5:
11     print("x는 10보다는 작거나 같고 5보다는 큼니다.")
12 else:
13     print("x가 5보다 작거나 같습니다.")
```


조건문 - 중첩

```
1  x = 5
2  y = 3
3
4  if x > 0:
5      |   if y > 0:
6          |       print("x와 y가 모두 양수입니다.")
7          |   else:
8          |       print("x는 양수지만 y는 음수입니다.")
9  else:
10     |   print("x는 음수입니다.")
```

```

1  from dynamic_gridworld import DynamicGridWorldEnv
2
3  # 환경 초기화
4  env = DynamicGridWorldEnv(size=5)
5
6  # 실행 흐름
7  env.reset() # 환경 초기화
8
9  # 여러 위치에서 물체 배치 및 확인
10 env.move_right() # 오른쪽으로 이동
11 env.place()      # 현재 위치에 물체 배치
12 env.move_left()  # 왼쪽으로 이동
13
14 # 주변 확인 및 행동 수행
15 senses = env.sense_all() # 네 방향 감지 결과 가져오기
16
17 if senses["right"]["box"]: # 오른쪽에 물체가 있는 경우
18     print("오른쪽에 물체가 있습니다!")
19     env.move_down()
20 elif senses["left"]["box"]: # 왼쪽에 물체가 있는 경우
21     print("왼쪽에 물체가 있습니다!")
22     env.move_up()
23 else: # 주변에 물체가 없는 경우
24     print("주변에 물체가 없습니다. 현재 위치에 물체를 배치합니다.")
25     env.place()
26
27 env.keep_window_open()

```

● (.venv) PS D:\marker-main> python .\2_if_elif_else.py
 오른쪽에 물체가 있습니다!
 Press 'ESC' or 'ENTER' to close the window to exit.

리스트 자료형 (list)

```
1  fruits = ['apple', 'banana', 'cherry']
```

반복문 - for

```
1  fruits = ['apple', 'banana', 'cherry']  
2  
3  for fruit in fruits:  
4      print(fruit)
```

반복문 - range

```
1  ✓ for i in range(5):  
2      |     print(i)  
3  
4      fruits = ['apple', 'banana', 'cherry']  
5  ✓ for i in range(3):  
6      |     print(fruits[i])  
7  
8  ✓ for fruit in fruits:  
9      |     print(fruit)
```

반복문 - while

```
1   count = 0
2
3   ✓ while count < 5:
4       |   print(count)
5       |   count += 1
6
7   ✓ for count in range(5):
8       |   print(count)
```

반복문 - while

```
1  # 사용자로부터 입력을 받고, 입력이 "quit"이면 반복을 종료
2  while True:
3      user_input = input("입력하세요: ")
4      if user_input == "quit":
5          break
6      print(user_input)
```

```

1  from dynamic_gridworld import DynamicGridWorldEnv
2
3  env = DynamicGridWorldEnv(size=5)
4
5  print("지그재그 패턴을 그립니다.")
6
7  while True:
8      # 오른쪽으로 이동하면서 물체 배치
9      for i in range(env.size - 1): # 오른쪽 끝까지 이동
10         env.place()
11         env.move_right()
12     env.place() # 오른쪽 끝에서 물체 배치
13
14     if not env.sense_down_wall(): # 아래쪽에 벽이 없으면 이동
15         env.move_down()
16     else: # 아래쪽에 벽이 있으면 루프 종료
17         print("아래쪽에 벽이 있습니다. 루프를 종료합니다.")
18         break
19
20     # 왼쪽으로 이동하면서 물체 배치
21     for i in range(env.size - 1): # 왼쪽 끝까지 이동
22         env.place()
23         env.move_left()
24     env.place() # 왼쪽 끝에서 물체 배치
25
26     if not env.sense_down_wall(): # 아래쪽에 벽이 없으면 이동
27         env.move_down()
28     else: # 아래쪽에 벽이 있으면 루프 종료
29         print("아래쪽에 벽이 있습니다. 루프를 종료합니다.")
30         break
31
32  env.keep_window_open()

```

```

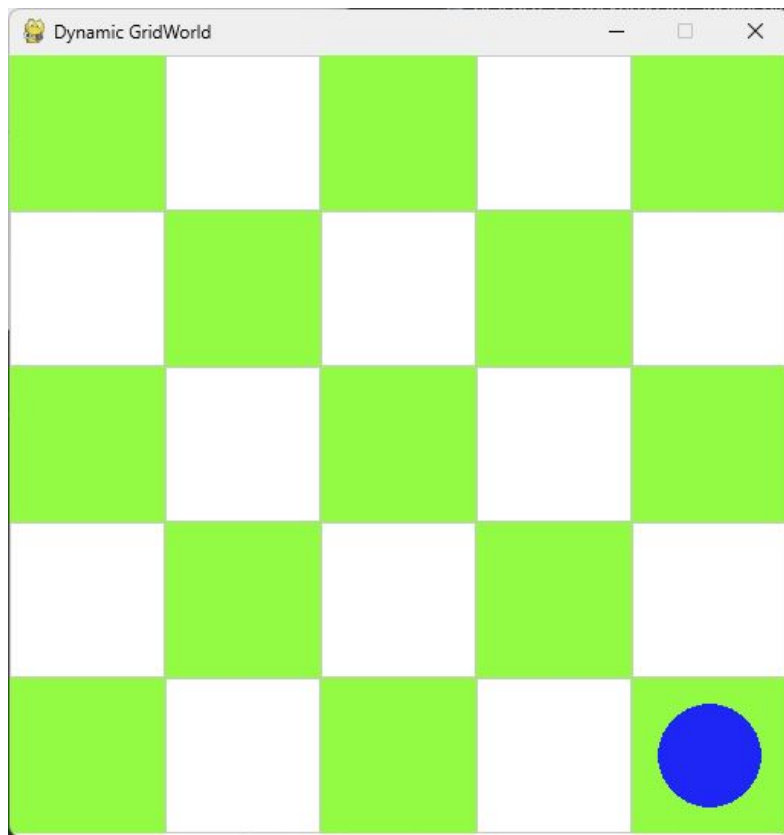
(.venv) PS D:\marker-main> python .\3_for_while.py
지그재그 패턴을 그립니다.
아래쪽에 벽이 있습니다. 루프를 종료합니다.
Press 'ESC' or 'ENTER' to close the window to exit.

```

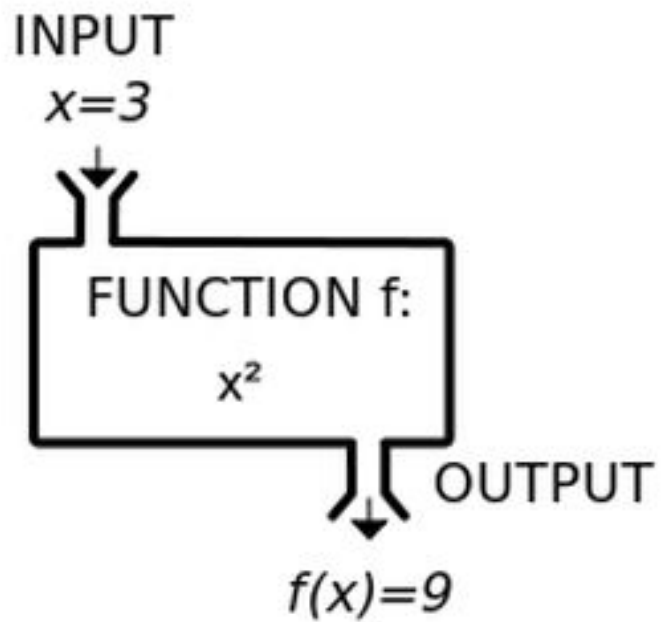

실습1 - 체크 보드 만들기

```
1  from dynamic_gridworld import DynamicGridWorldEnv
2
3  env = DynamicGridWorldEnv(size=5)
4
5  print("체크무늬 패턴을 그리니다.")
6
7  # 체크무늬 패턴 생성
8  for row in range(env.size):
9      for col in range(env.size):
10         ##### 여기 아래에서 코드를 작성해주세요 #####
11
12
13
14
15
16
17
18
19
20
21
22         #####
23
24  env.keep_window_open()
```

실습1 - 체크 보드 만들기



함수



함수

```
1  def square(x):  
2      |    return x**2  
3  
4  input_value = 3  
5  output_value = square(input_value)  
6  
7  print(f'함수의 실행 결과 f(x)는 {output_value} 입니다.')  
8  # 함수의 실행 결과 f(x)는 9 입니다.
```

함수 - 인자와 반환값

```
1  def add_order_result(user_id, product_id, final_price, quantity):  
2  
3      # do somethings...  
4  
5      return 0
```

함수 - 사용

```
1  def is_prime(number):
2      """주어진 숫자가 소수인지 여부를 판별하는 함수"""
3      if number <= 1:
4          return False
5      if number <= 3:
6          return True
7      if number % 2 == 0 or number % 3 == 0:
8          return False
9
10     i = 5
11     while i * i <= number:
12         if number % i == 0 or number % (i + 2) == 0:
13             return False
14         i += 6
15
16     return True
```

함수 - 사용

```
1 > def is_prime(number): ...
3
4 result = is_prime(16)
5 if result:
6     print("16은 소수입니다.")
7 else:
8     print("16은 소수가 아닙니다.")
9 # 16은 소수가 아닙니다.
10
11 result = is_prime(17)
12 if result:
13     print("17은 소수입니다.")
14 else:
15     print("17은 소수가 아닙니다.")
16 # 17은 소수입니다.
```

다양한 타입 - 딕셔너리 (Dictionary)

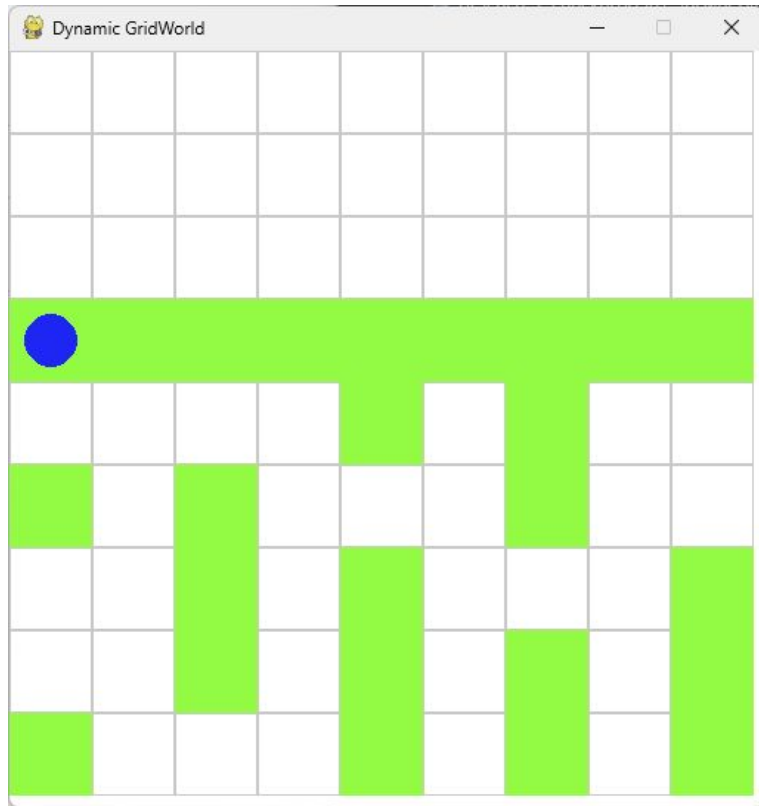
```
1  # 딕셔너리 생성
2  student = {"name": "Alice", "age": 25, "courses": ["Math", "Science"]}
3
4  # 값 조회
5  print(student["name"]) # 출력: Alice
6
7  # 값 추가 및 수정
8  student["age"] = 26
9  student["grade"] = "A"
10
11 # 값 삭제
12 del student["courses"]
13
14 print(student) # 출력: {'name': 'Alice', 'age': 26, 'grade': 'A'}
```


다양한 타입 - 튜플 (Tuple)

```
1  # 튜플 생성
2  coordinates = (10.0, 20.0)
3
4  # 요소 접근
5  print(coordinates[0]) # 출력: 10.0
6
7  # 불변성: 튜플은 요소를 변경할 수 없습니다.
8  coordinates[0] = 15.0 # 오류 발생
```

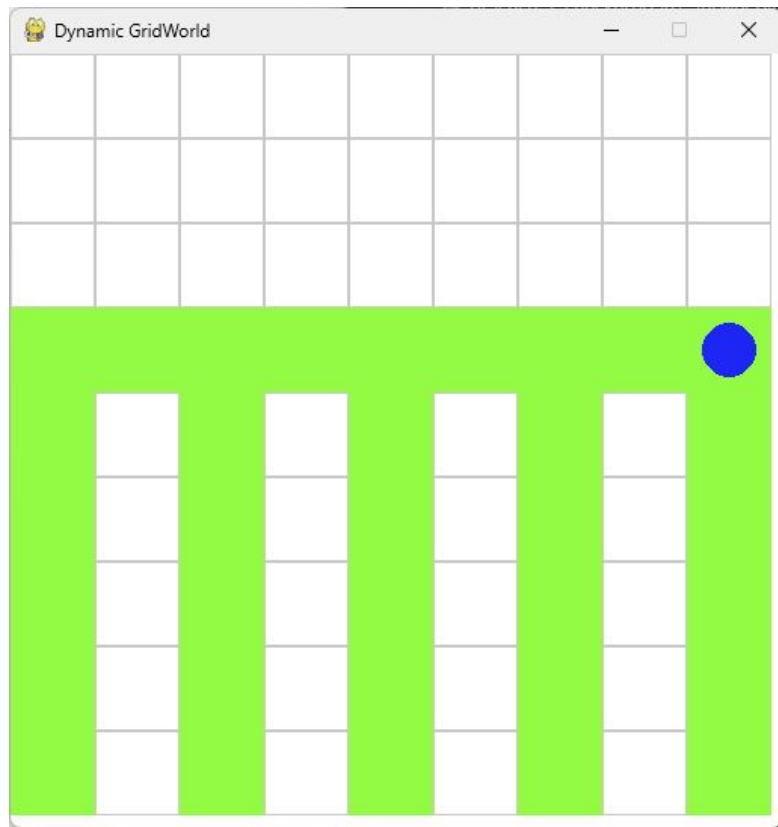
실습2 - 다리 수리 하기

```
1 from dynamic_gridworld import DynamicGridWorldEnv
2
3 # 환경 초기화 및 깨진 다리 로드
4 env = DynamicGridWorldEnv(size=9)
5 env.reset()
6 env.load_broken_bridge()
7
8 # 다리 수리 함수 정의
9 def fix_pole():
10     #####
11
12
13
14
15
16
17
18
19
20
21
22
23     #####
24
25 # 전체 다리 수리
26 for _ in range(9): # 8칸 이동하며 수리
27     y = env.agent_pos[1] # 현재 y 좌표 저장
28     # 현재 위치의 y좌표가 홀수일 때만 수리 진행
29     if y % 2 == 0:
30         fix_pole()
31     env.move_right()
32
33 env.keep_window_open()
```



실습2 - 다리 수리 하기

- 다음중에서 하나를 써보세요.
 - `sense('down')`
 - `sense_down()`
 - `sense_box('down')`
 - `sense_down_box()`
 - `sense_wall('down')`
 - `sense_down_wall()`



파이썬 학습을 위해 도움이 되는 사이트

1. 점프 투 파이썬(<https://wikidocs.net/book/1>)
2. 코딩도장 (<https://dojang.io/course/view.php?id=7>)
3. 프로그래머스 실습 (<https://school.programmers.co.kr/learn/challenges>)
 - 다양한 난이도의 연습문제 제공
 - 다른 사람의 정답을 보고 학습 가능
4. 백준 온라인 저지 (<https://www.acmicpc.net/>)
 - 알고리즘 문제 풀이 사이트
 - 중급 이상의 실력을 가진 사용자들에게 추천

파이썬 학습을 위해 도움이 되는 사이트

1. Google's Python Class (<https://developers.google.com/edu/python>)
2. Codecademy (<https://www.codecademy.com/catalog/language/python>)
3. Codewars (<https://www.codewars.com/>)
 - 게임화된 방식으로 코딩 문제를 풀 수 있는 사이트
4. LeetCode (<https://leetcode.com/problemset/>)
 - 코딩 테스트와 인터뷰 준비에 특화된 사이트
5. Codeforces (<https://codeforces.com/>)
 - 중급 이상 파이썬 사용자를 위한 알고리즘 문제 풀이 사이트

감사합니다